CS 306: Error Detection and Correction

Dr. Ayan Mondal

Assistant Professor

Dept. of CSE, IIT Indore

https://www.iiti.ac.in/people/~ayanm/

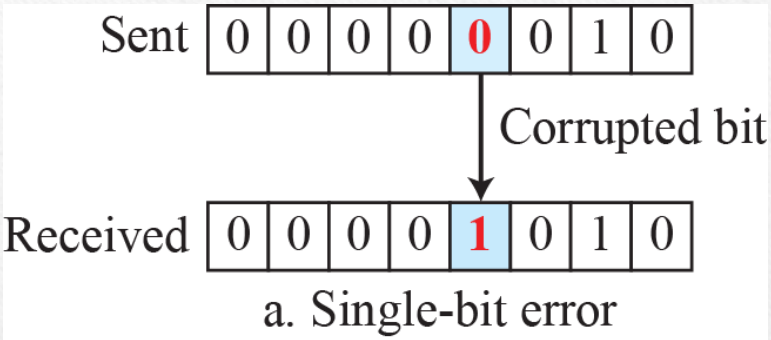
Outline

- Block coding
- Cyclic Codes
- Checksum
- Forward error correction

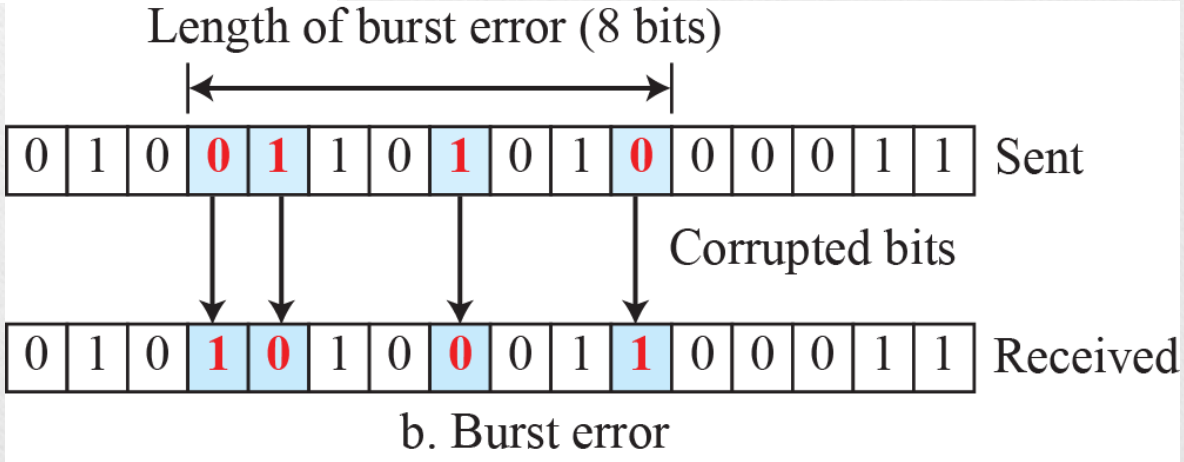
Types of Errors

- Whenever bits flow from one point to another, they are subject to unpredictable changes because of interference. This interference can change the shape of the signal.
- The term single-bit error means that only 1 bit of a given data unit is changed
- The term burst error means that 2 or more bits in the data unit have changed

Single-bit and burst error



a. Single-bit error



b. Burst error

Redundancy

- The central concept in detecting or correcting errors is redundancy.
- To be able to detect or correct errors, we need to send some extra bits with our data.
- These redundant bits are added by the sender and removed by the receiver.
- Their presence allows the receiver to detect or correct corrupted bits.

Detection vs Correction

- In error detection, we are only looking to see if any error has occurred. The answer is a simple yes or no. We are not even interested in the number of corrupted bits. A single-bit error is the same for us as a burst error.
- In error correction, we need to know the exact number of bits that are corrupted and, more importantly, their location in the message.

Coding

- Redundancy is achieved through various coding schemes.
- The sender adds redundant bits through a process that creates a relationship between the redundant bits and the actual data bits. The receiver checks the relationships between the two sets to detect errors.
- The ratio of redundant bits to data bits and the robustness of the process are important factors in any coding scheme.

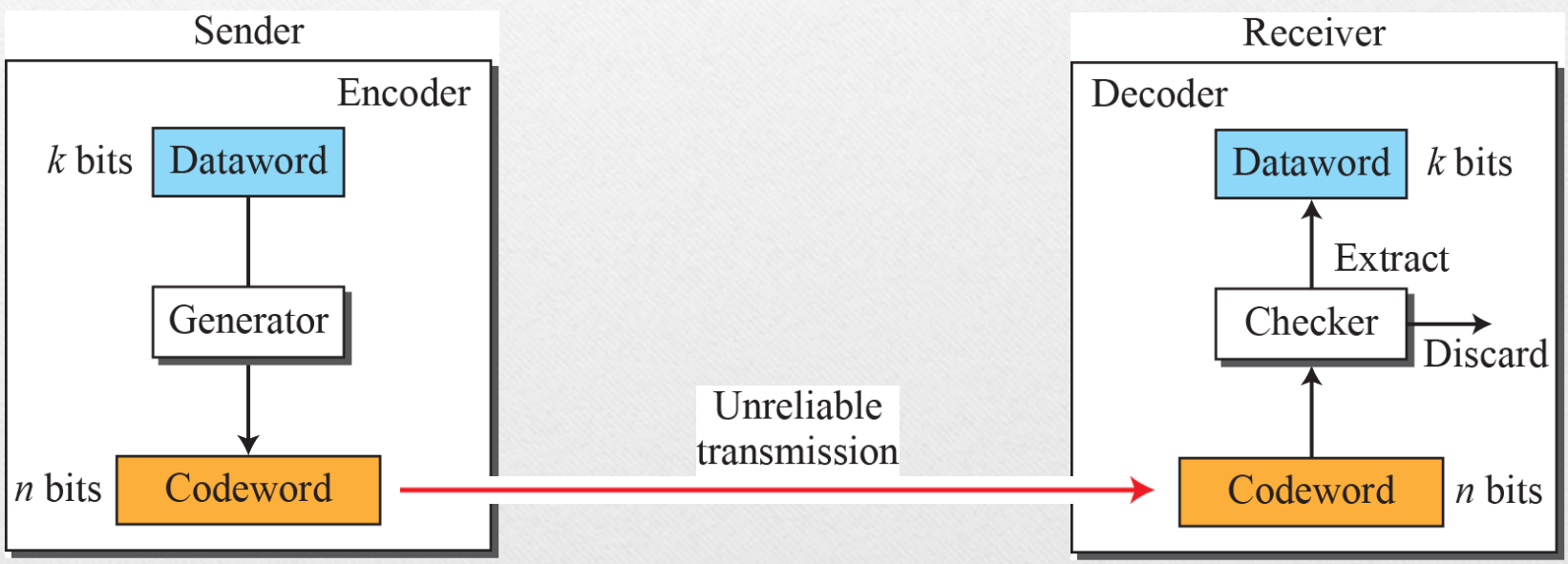
Block Coding

- In block coding, we divide our message into blocks, each of k bits, called datawords.
- We add r redundant bits to each block to make the length $n = k + r$.
- The resulting n-bit blocks are called codewords.

Error Detection

- If the following two conditions are met, the receiver can detect a change in the original codeword.
  - The receiver has (or can find) a list of valid codewords.
  - The original codeword has changed to an invalid one.

Error Detection (Contd.)



Example

- Let us assume that k = 2 and n = 3. The list of datawords and codewords are as follows:

Datawords	*Codewords*	*Datawords*	*Codewords*
00	000	10	101
01	011	11	110

Assume the sender encodes the dataword 01 as 011 and sends it to the receiver. Consider the following cases:

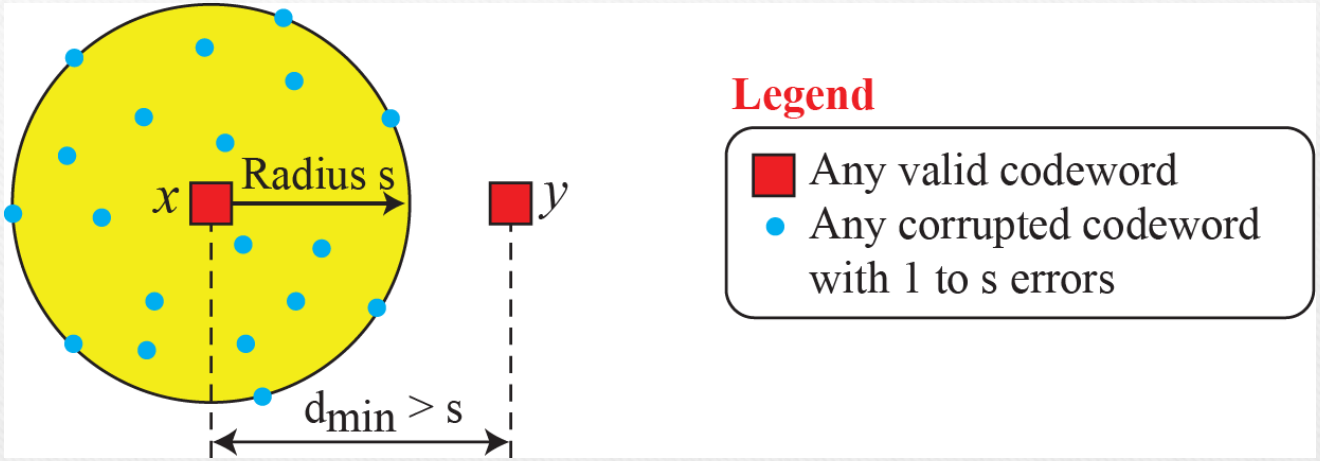
1. The receiver receives 011. It is a valid codeword. The receiver extracts the dataword 01 from it.
2. The codeword is corrupted during transmission, and 111 is received (the leftmost bit is corrupted). This is not a valid codeword and is discarded.
3. The codeword is corrupted during transmission, and 000 is received (the right two bits are corrupted). This is a valid codeword. The receiver incorrectly extracts the dataword 00. Two corrupted bits have made the error undetectable.

Hamming Distance

- Hamming distance is a metric for comparing two binary data strings.
- While comparing two binary strings of equal length, Hamming distance is the number of bit positions in which the two bits are different.

1. The Hamming distance $d(000, 011)$ is 2 because $(000 \oplus 011)$ is 011 (two 1s).
2. The Hamming distance $d(10101, 11110)$ is 3 because $(10101 \oplus 11110)$ is 01011 (three 1s).

Geometric concept explaining $d_{\min}$ in error detection



A code scheme has a Hamming distance $d_{\min}$. This code guarantees the detection of up to **s** errors, where $\boldsymbol{d_{min}} = \mathbf{s} + \mathbf{1}$

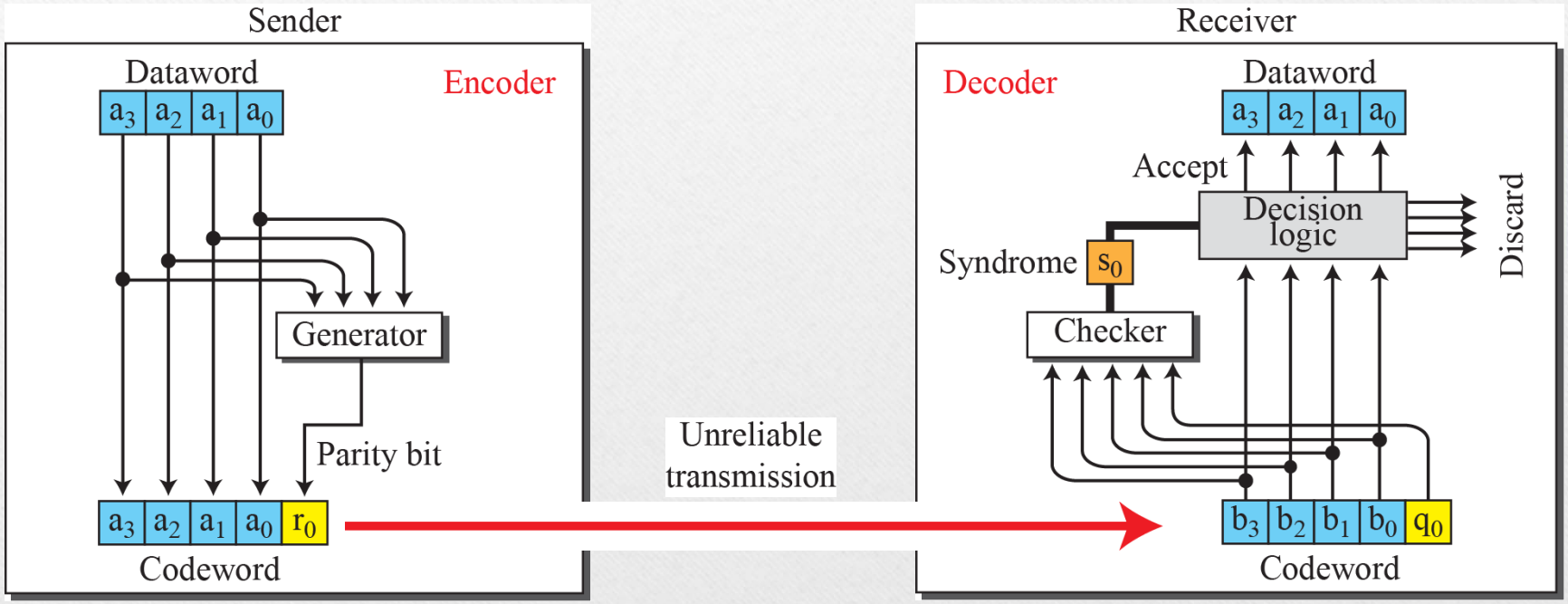
Example

- The minimum Hamming distance for first example is 2. It guarantees detection of only a single error.
- If the third codeword (101) is sent and one error occurs, the received codeword does not match any valid codeword.
- If two errors occur, however, the received codeword may match a valid codeword and the errors are not detected.

Simple parity-check code C(5, 4)

Datawords	**Codewords**	*Datawords*	**Codewords**
0000	**00000**	1000	**10001**
0001	**00011**	1001	**10010**
0010	**00101**	1010	**10100**
0011	**00110**	1011	**10111**
0100	**01001**	1100	**11000**
0101	**01010**	1101	**11011**
0110	**01100**	1110	**11101**
0111	**01111**	1111	**11110**

Encoder and decoder for simple parity-check code



Modular Arithmetic (Modulo-2) operation

Cyclic Code

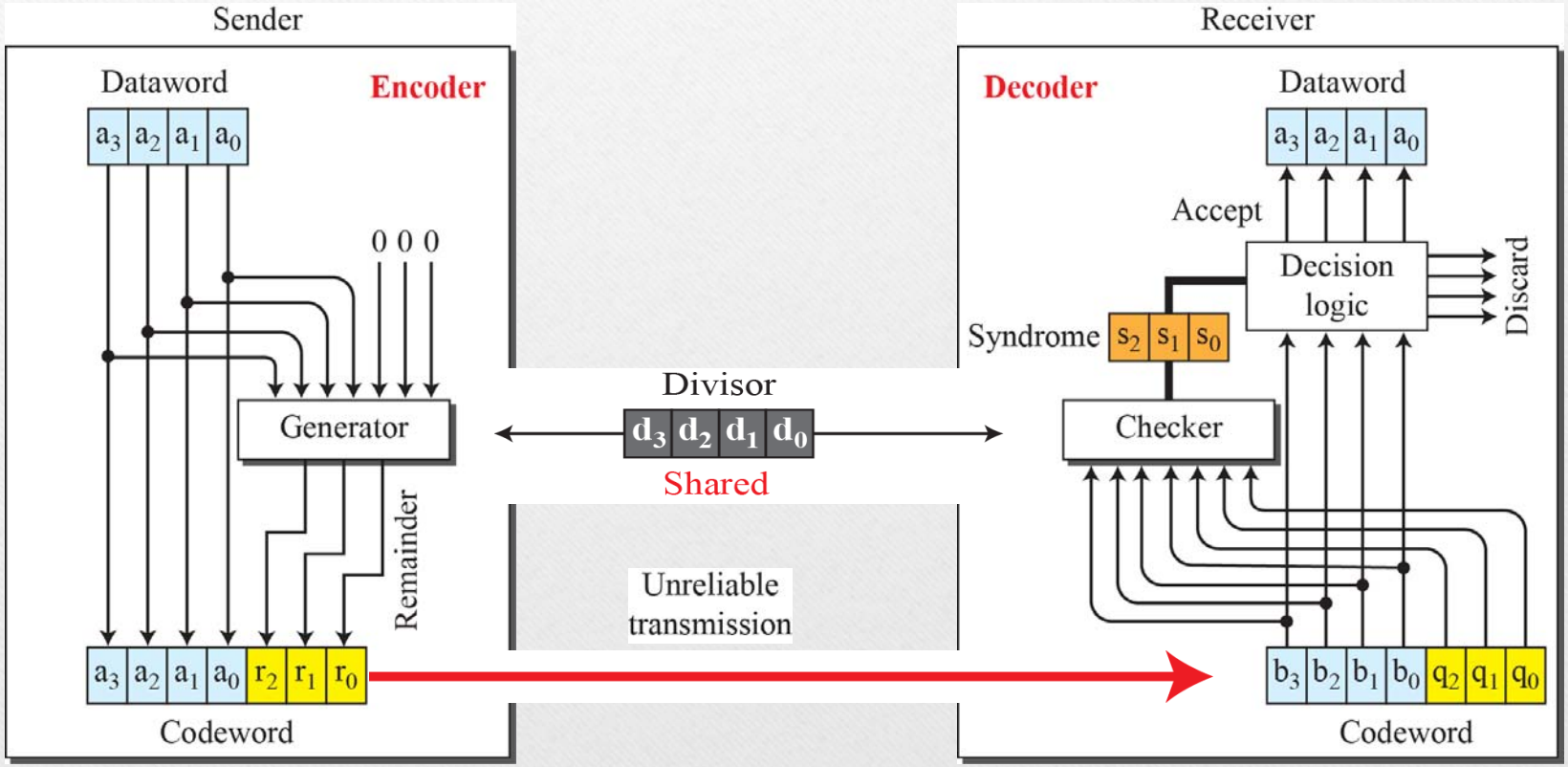
- Cyclic codes are special linear block codes with one extra property.
- In a cyclic code, if a codeword is cyclically shifted (rotated), the result is another codeword.
- For example, if 1011000 is a codeword and we cyclically left-shift, then 0110001 is also a codeword.

Cyclic Redundancy Check

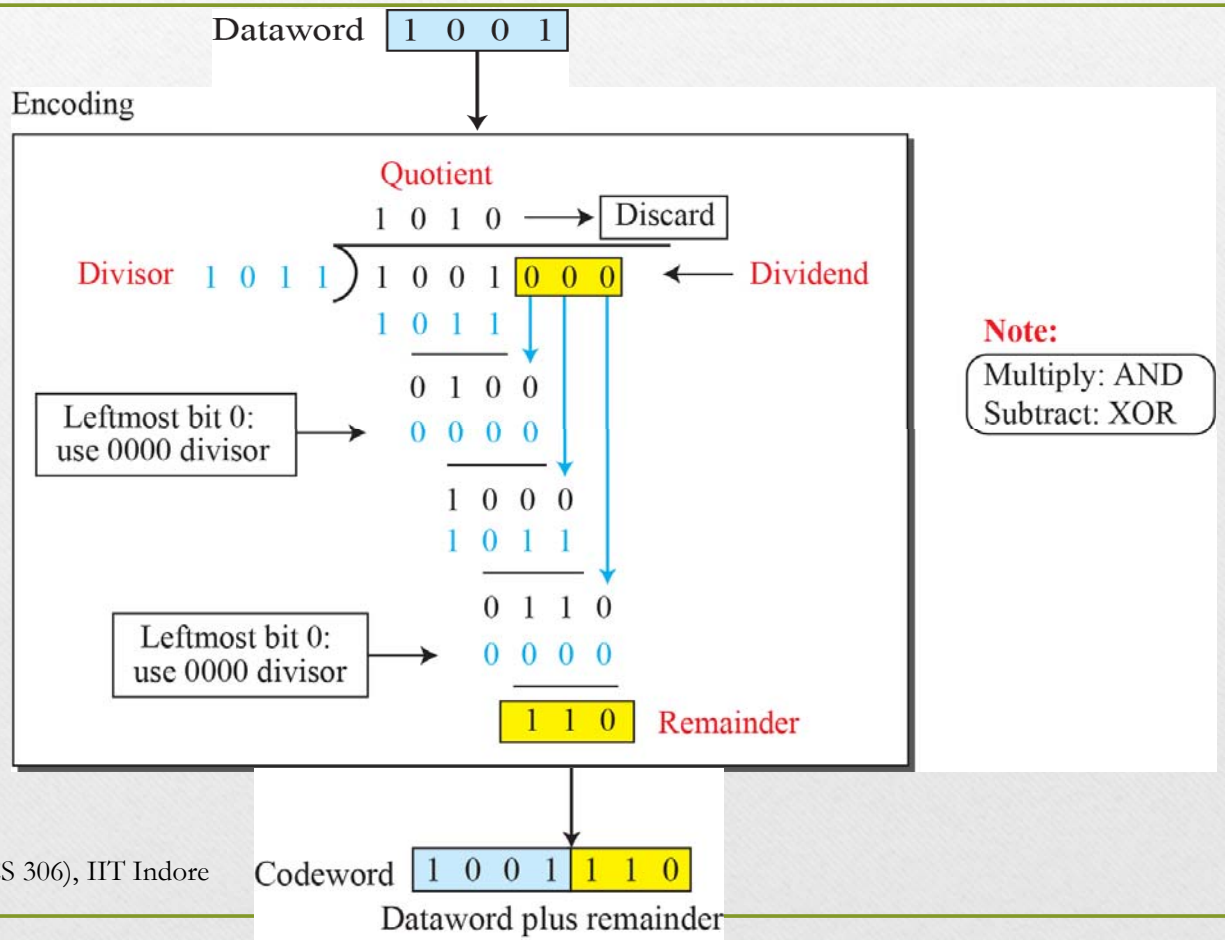
- A subset of. cyclic codes is called the cyclic redundancy check (CRC), which is used in networks such as LANs and WANs.

Dataword	*Codeword*	*Dataword*	*Codeword*
0000	0000000	1000	1000101
0001	0001011	1001	1001110
0010	0010110	1010	1010011
0011	0011101	1011	1011000
0100	0100111	1100	1100010
0101	0101100	1101	1101001
0110	0110001	1110	1110100
0111	0111010	1111	1111111

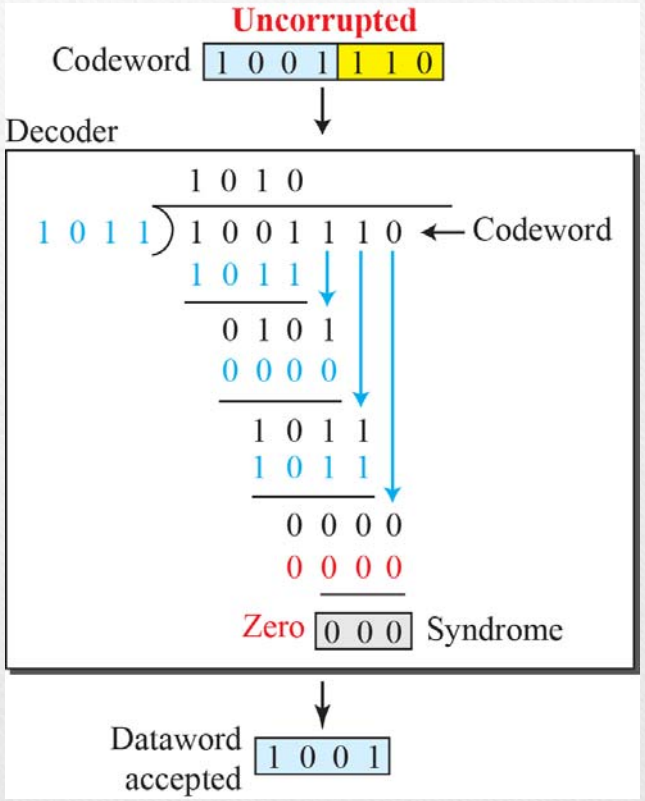
CRC encoder and decoder

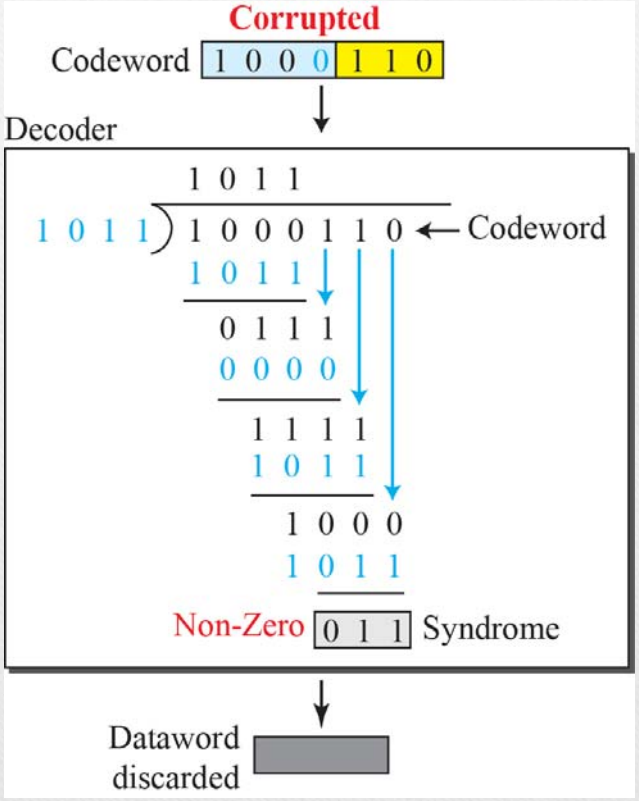


Division in CRC encoder



Division in the CRC decoder for two cases

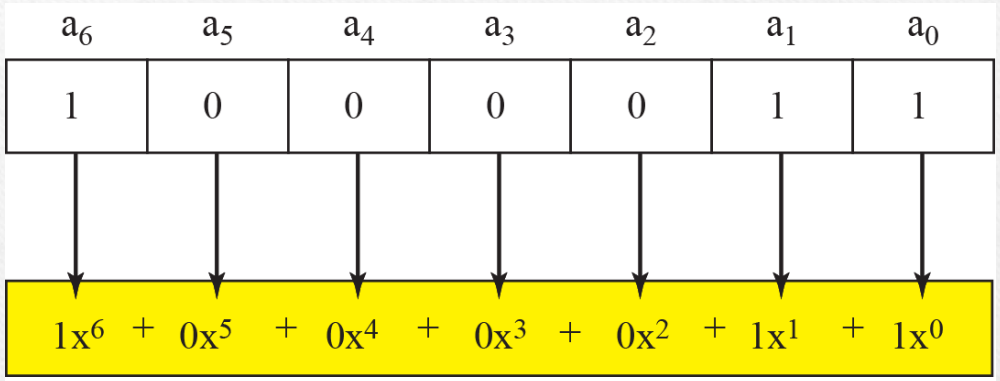




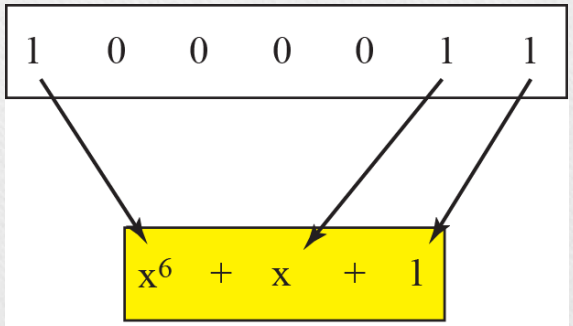
Polynomials

- A better way to understand cyclic codes and how they can be analyzed is to represent them as polynomials.
- A pattern of 0s and 1s can be represented as a polynomial with coefficients of 0 and 1.
- The power of each term shows the position of the bit; the coefficient shows the value of the bit.

A polynomial to represent a binary word

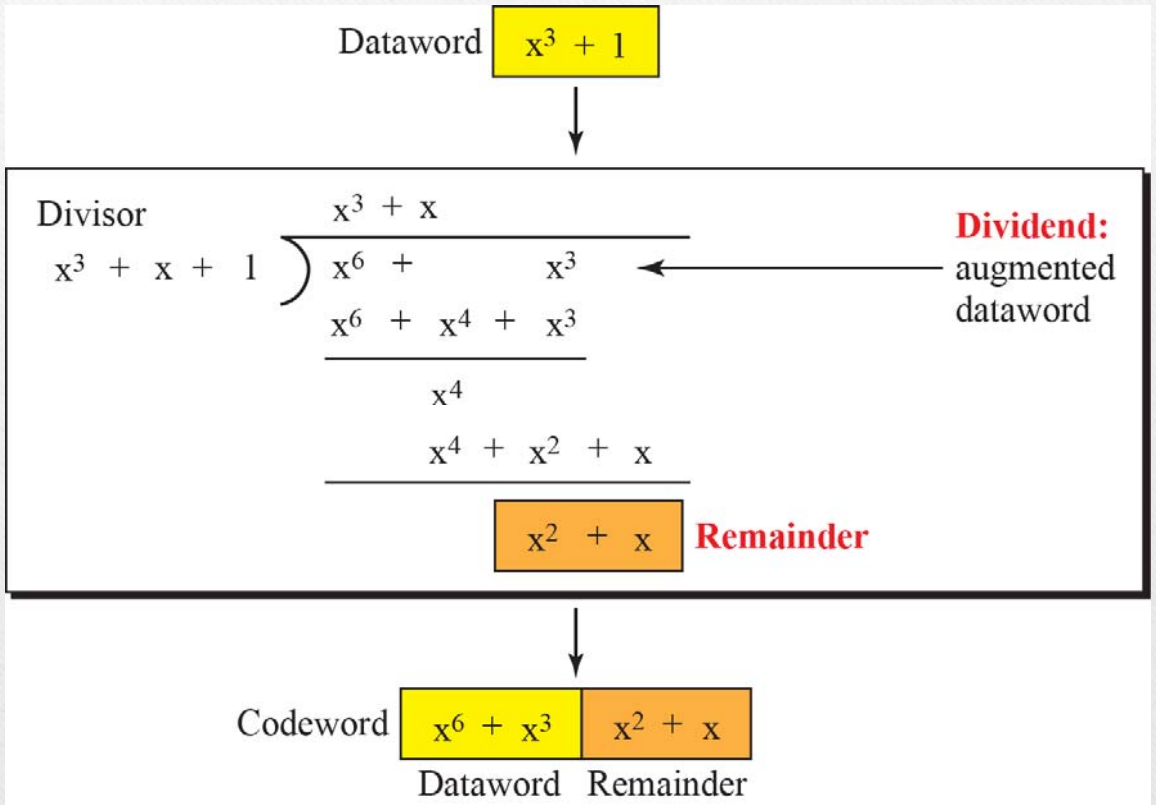


a. Binary pattern and polynomial



b. Short form

Encoder Using Polynomials



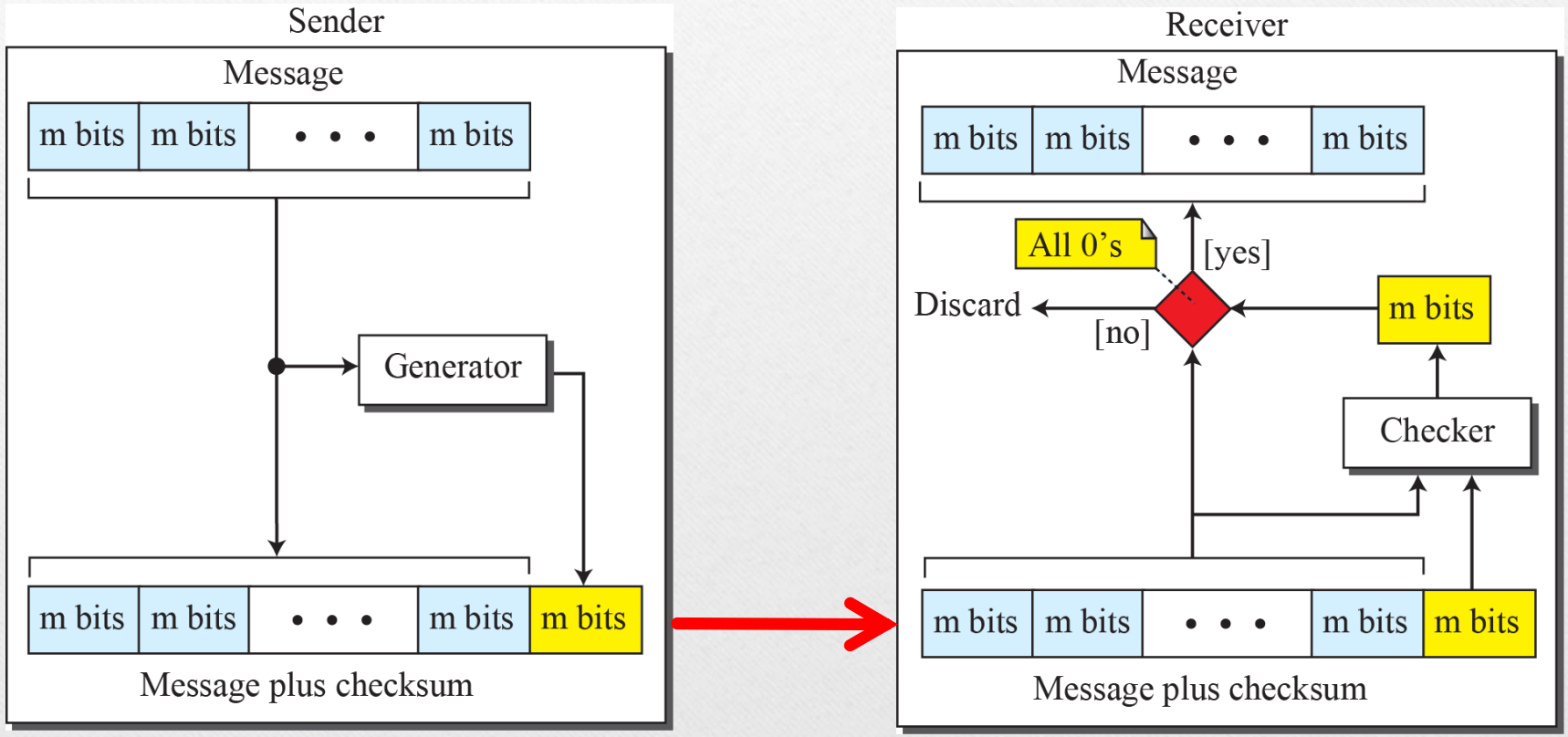
Standard polynomials

Name	*Polynomial*	*Used in*
CRC-8	$x^8 + x^2 + x + 1$ **100000111**	ATM header
CRC-10	$x^{10} + x^9 + x^5 + x^4 + x^2 + 1$ **11000110101**	ATM AAL
CRC-16	$x^{16} + x^{12} + x^5 + 1$ **10001000000100001**	HDLC
CRC-32	$x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$ **100000100110000010001110110110111**	LANs

Checksum

- Checksum is an error-detecting technique that can be applied to a message of any length.
- In the Internet, the checksum technique is mostly used at **the network and transport layer** rather than the data-link layer.

Checksum (Contd.)



Example

- Suppose the message is a list of five 4-bit numbers that we want to send to a destination. In addition to sending these numbers, we send the sum of the numbers.
- For example, if the set of numbers is (7, 11, 12, 0, 6), we send (7, 11, 12, 0, 6, 36). The receiver adds the five numbers and compares the result with the sum.
- If these are the same, the receiver assumes no error, accepts the five numbers, and discards the sum.

Example (Contd.)

- In the previous example, the decimal number 36 in binary is (100100)2. To change it to a 4-bit number we add the extra leftmost bit to the right four bits as shown below.

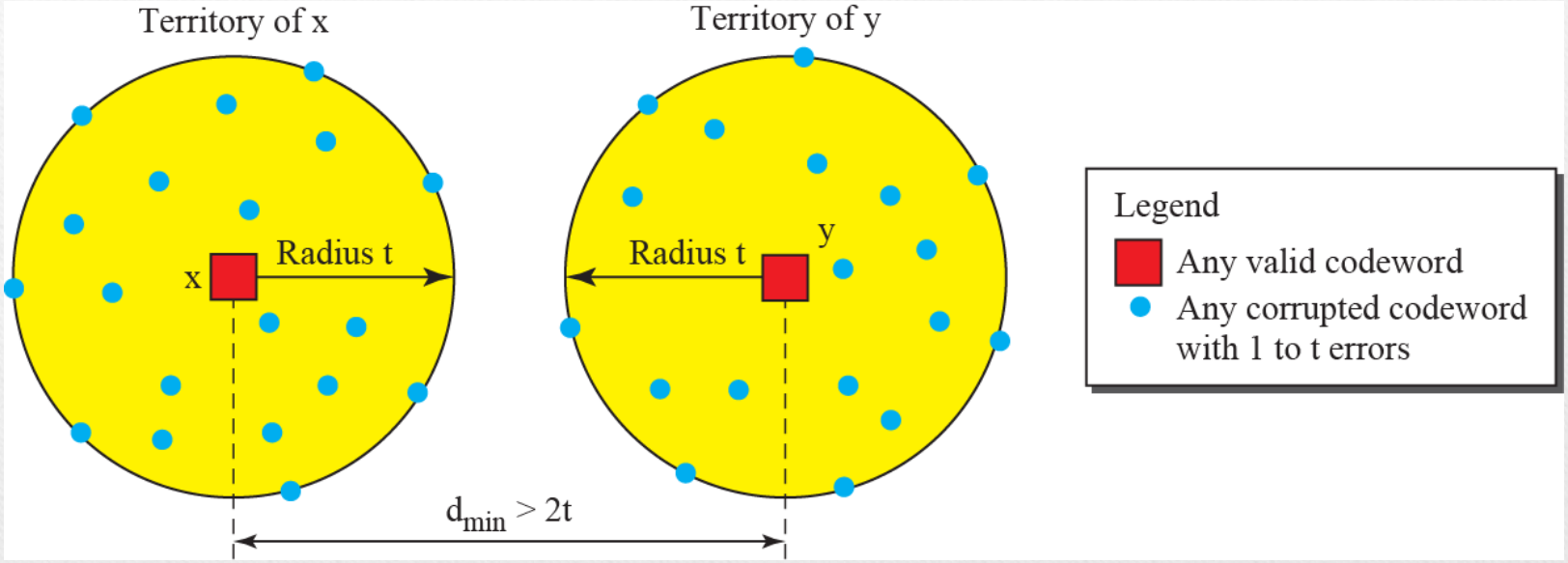
$$(10)_2 + (0100)_2 = (0110)_2 \rightarrow (6)_{10}$$

- Instead of sending 36 as the sum, we can send 6 as the sum (7, 11, 12, 0, 6, 6).

Forward Error Correction

- For error detection, we definitely need more distance.
- It can be shown that to detect t errors, we need to have $d_{min} = 2t + 1$.
- In other words, if we want to correct 10 bits in a packet, we need to make the minimum hamming distance 21 bits, which means a lot of redundant bits need to be sent with the data.

Hamming distance for error correction



Next Class

- Data Link Control

?

ayanm@iiti.ac.in

Room No. 407, POD-1A